



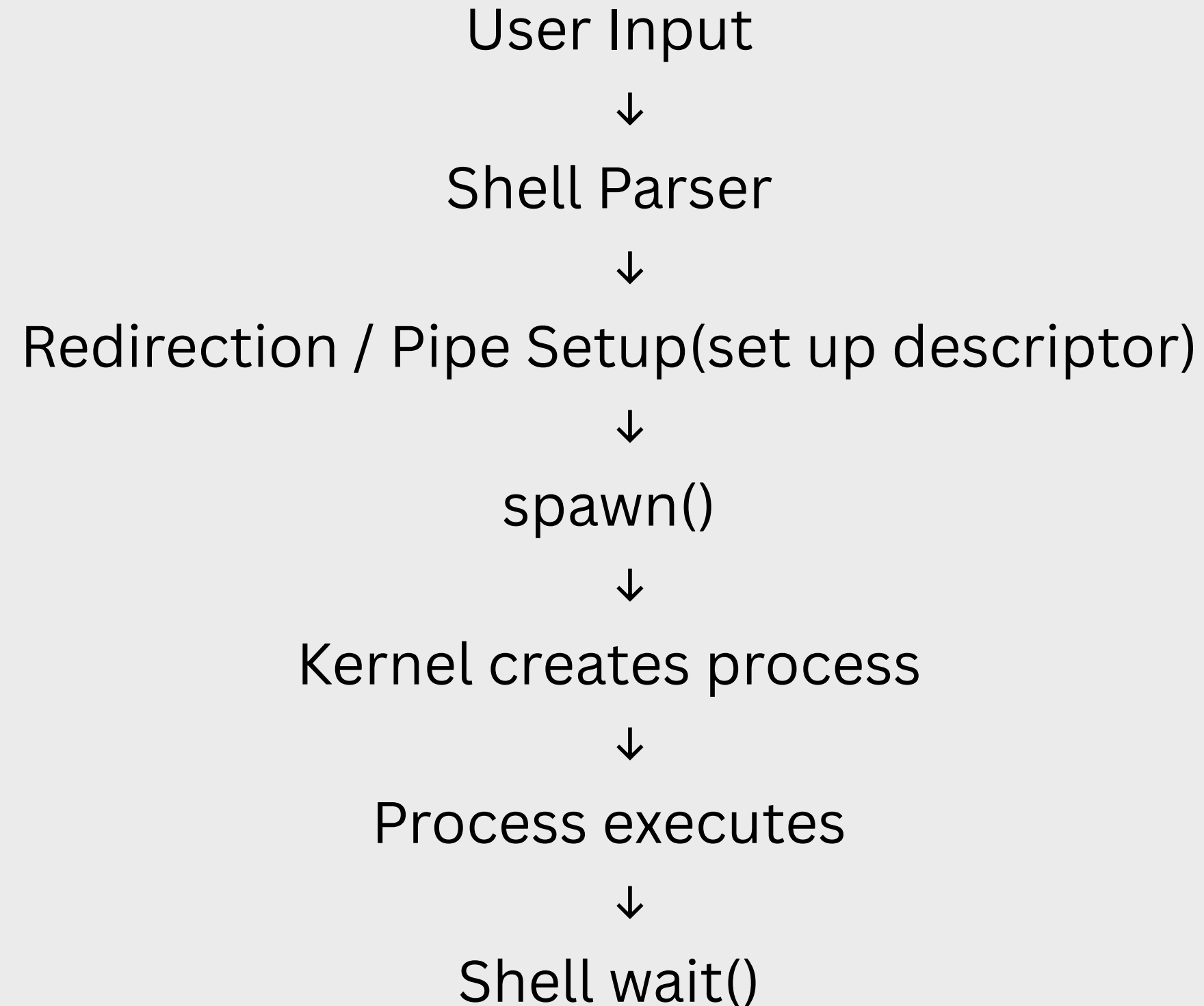
UNIX SHELL

Group 09

OBJECTIVE

- Implement a UNIX-style interactive shell
- Support file and directory operations
- Implement file descriptor abstraction
- Enable I/O redirection (<, >, >>)
- Implement pipelines (|)
- Support recursive directory operations
- Maintain process isolation

SYSTEM ARCHITECTURE OVERVIEW



SYSTEM ARCHITECTURE OVERVIEW

User Space:

- **Shell process** – reads commands, manages environment, handles I/O redirection.
- **Command implementations** (cp, mv, rm, etc.) – perform file operations using system calls.
- **Command parser** – splits input into command + arguments, handles options, pipes, and redirection.

SYSTEM ARCHITECTURE OVERVIEW

Kernel Space

- **File system** (inodes + directories) – manages files, directories, and metadata.
- **File descriptor table** – tracks open files, positions, and access modes.
- **Pipe subsystem** – enables communication between processes.
- **System call handler** – validates user requests and executes syscalls (open, read, write, etc.).

MAJOR MODULES

THE PARSER (THE BRAIN)

Reading input - Grabs text using `fgets()` and stores safely in a buffer

Breaking it down - Tokenize each command and argument using `strtok()`

Spotting special symbols - Spots symbols like `>` or `|` to redirect data or link processes

Cleaning up - Makes sure the shell doesn't leak memory or crash after a few commands

THE EXECUTOR (THE WORKER)

Spawning Processes - Uses `fork()` to create child process every time a command runs

Running the Program - Uses `execvp()` to flip the child process from shell copy into being the actual program

Finding the File - Looks through the system's path automatically

Staying in Sync - Uses `waitpid()` to make the parent shell wait until the command is finished

BUILT-IN COMMANDS (THE INSIDERS)

Staying Local - Commands like `cd`/`exit` are handled by shell itself without creating a process

Changing Direction - Uses `chdir()` to move shell's current location

Fast & Simple - Since there's no `fork()` or `exec()` involved, these commands are nearly instant

Shutting Down - the `exit` command is built-in so the shell can clean up its own memory and close down properly

I/O & PIPES (THE DATA FLOW)

Rerouting Output - Uses `dup2()` to send output to a file instead of screen when it sees `>`

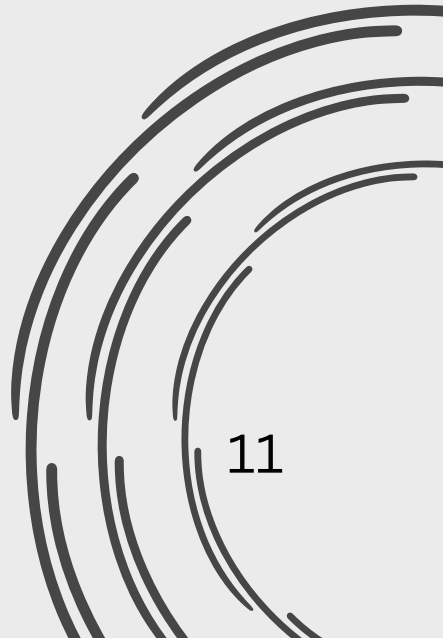
Connecting Commands - Sets up `pipe()` which lets data flow from one command to another

Housekeeping - Carefully closes old file descriptors to keep the system organized

Handling the Chain - It manages several processes at once to make sure a long pipe runs smoothly



IMPLEMENTATION



ls

mcertikos/kern/trap/TSyscall/TSyscall.c

```
void sys_ls(tf_t *tf)
{
    uint32_t off, inum;
    struct dirent de;
    uint32_t de_size;
    struct inode * p_inode;
    int user_buf_addr = syscall_get_arg2(tf);
    int buf_len = syscall_get_arg3(tf);
    int cur_pid = get_curid();
    int len;
    char * buf_p = sys_buf[cur_pid];
    struct inode* dp = (struct inode*)tcb_get_cwd(cur_pid);

    // Lazily initialize cwd to root if not set
    if (dp == NULL) {
        dp = inode_get(ROOTDEV, ROOTINO);
        tcb_set_cwd(cur_pid, dp);
    }

    // Lock the inode to ensure it's valid
    inode_lock(dp);

    de_size = sizeof(de);
    for(off = 0; off < dp->size; off += de_size){
        if(inode_read(dp, (char *)&de, off, de_size) != de_size){
            // size is not legal
            inode_unlock(dp);
            KERN_PANIC("wrong in dir_lookup");
        }
        if(de.inum == 0){
            // free entry
            continue;
        }
        Implement with Codex
        // TODO index may out of bound
        strncpy(buf_p, de.name, strlen(de.name, PAGESIZE));
        buf_p += strlen(de.name, PAGESIZE);
        *(buf_p++) = ' ';
        //dprintf("%s ", de.name);
    }

    inode_unlock(dp);

    *(buf_p - 1) = '\0';
    //dprintf("\n");
    len = buf_p - sys_buf[cur_pid] < buf_len? buf_p - sys_buf[cur_pid]: buf_len;
    pt_copyout(sys_buf[cur_pid], cur_pid, user_buf_addr, len);
    syscall_set_errno(tf, E_SUCC);
    syscall_set_retval1(tf, len);
}
```

cd

mcertikos/kern/fs/sysfile.c

```
void sys_chdir(tf_t *tf)
{
    char path[128];
    struct inode *ip;
    int pid = get_curid();
    unsigned int path_len = syscall_get_arg3(tf);
    path_len = path_len < 128? path_len: 127;
    pt_copyin(get_curid(), syscall_get_arg2(tf), path, path_len);
    path[path_len] = '\0';

    if((ip = namei(path)) == 0){
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }
    inode_lock(ip);
    if(ip->type != T_DIR){
        inode_unlockput(ip);
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }
    inode_unlock(ip);
    inode_put(tcb_get_cwd(pid));
    tcb_set_cwd(pid, ip);
    syscall_set_errno(tf, E_SUCC);
}
```



pwd

mcertikos/kern/trap/TSyscall/TSyscall.c

```
void sys_pwd(tf_t *tf)
{
    char names[64][DIRSIZ + 1];
    char path[512];
    int depth = 0;
    int pid = get_curid();
    struct inode* curi = (struct inode*)tcb_get_cwd(pid);
    struct inode* parent;
    struct dirent de;
    unsigned int poff, off;
    unsigned int de_size = sizeof(struct dirent);
    char *p = path;

    // Lazily initialize cwd to root if not set.
    if (curi == NULL) {
        curi = inode_get(ROOTDEV, ROOTINO);
        tcb_set_cwd(pid, curi);
    }
    curi = inode_dup(curi); // local reference while walking

    while (depth < 64) {
        int found = 0;
        inode_lock(curi);
        if (curi->type != T_DIR) {
            inode_unlock(curi);
            break;
        }
        parent = dir_lookup(curi, "..", &poff);
        inode_unlock(curi);
        if (parent == 0) {
            break;
        }
        inode_lock(parent);
        if (parent->type != T_DIR) {
            inode_unlock(parent);
            inode_put(parent);
            break;
        }
        // Reached root when "." points to itself.
        if (parent->inum == curi->inum) {
            inode_unlock(parent);
            inode_put(parent);
            break;
        }
    }
```

```
        for (off = 0; off < parent->size; off += de_size) {
            if (inode_read(parent, (char *)&de, off, de_size) != de_size)
                break;
            if (de.inum == curi->inum) {
                strncpy(names[depth], de.name, DIRSIZ);
                names[depth][DIRSIZ] = '\0';
                depth++;
                found = 1;
                break;
            }
        }
        inode_unlock(parent);
        if (!found) {
            inode_put(parent);
            break;
        }
        inode_put(curi);
        curi = parent;
    }
    inode_put(curi);
    if (depth == 0) {
        path[0] = '/';
        path[1] = '\0';
    } else {
        for (int i = depth - 1; i >= 0; i--) {
            *p++ = '/';
            strncpy(p, names[i], DIRSIZ);
            p += strlen(names[i], DIRSIZ);
        }
        *p = '\0';
    }
    pt_copyout(path, pid, syscall_get_arg2(tf), strlen(path, sizeof(path)) + 1);
    syscall_set_errno(tf, E_SUCC);
    syscall_set_retval(tf, 0);
}
```

mkdir

mcertikos/kern/fs/sysfile.c

```
void sys_mkdir(tf_t *tf)
{
    char path[128];
    struct inode *ip, *dp;
    char name[DIRSIZ];
    unsigned int path_len;
    static int first = TRUE;
    if (first) {
        first = FALSE;
        log_init();
    }

    // Copy path from user
    path_len = syscall_get_arg3(tf);
    path_len = path_len < 128 ? path_len : 127;
    pt_copyin(get_curid(), syscall_get_arg2(tf), path, path_len);
    path[path_len] = '\0';

    // Find parent directory
    if ((dp = nameiparent(path, name)) == 0) {
        syscall_set_errno(tf, E_NEXIST);
        return;
    }

    begin_trans();

    inode_lock(dp);

    // Allocate the directory inode
    if ((ip = inode_alloc(dp->dev, T_DIR)) == 0) {
        inode_unlockput(dp);
        commit_trans();
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }

    inode_lock(ip);
    ip->nlink = 1;      // For "."
    inode_update(ip);  // Write inode metadata
```

```
    // Add "." entry
    if (dir_link(ip, ".", ip->inum) < 0) {
        inode_unlockput(ip);
        inode_unlockput(dp);
        commit_trans();
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }

    // Add ".." entry
    if (dir_link(ip, "..", dp->inum) < 0) {
        inode_unlockput(ip);
        inode_unlockput(dp);
        commit_trans();
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }

    inode_unlock(ip);

    // Increase parent nlink for ".."
    dp->nlink++;
    inode_update(dp);

    // Link new dir into parent
    if (dir_link(dp, name, ip->inum) < 0) {
        inode_unlockput(dp);
        inode_put(ip);
        commit_trans();
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }

    inode_unlockput(dp);
    inode_put(ip);

    commit_trans();

    syscall_set_errno(tf, E_SUCC);
}
```

touch

mcertikos/kern/fs/sysfile.c

```
static struct inode*
create(char *path, short type, short major, short minor)
{
    struct inode *ip, *dp;
    char name[DIRSIZ];
    uint32_t off;

    if((dp = nameiparent(path, name)) == 0)
        return 0;
    inode_lock(dp);

    // FIXED: dir_lookup instead of dirlookup
    if((ip = dir_lookup(dp, name, &off)) != 0){
        inode_unlockput(dp);
        inode_lock(ip);
        if(type == T_FILE && ip->type == T_FILE)
            return ip;
        inode_unlockput(ip);
        return 0;
    }

    if((ip = inode_alloc(dp->dev, type)) == 0)
        KERN_PANIC("create: inode_alloc");

    inode_lock(ip);
    ip->major = major;
    ip->minor = minor;
    ip->nlink = 1;
    inode_update(ip);

    if(type == T_DIR){
        dp->nlink++;
        inode_update(dp);

        // FIXED: dir_link instead of dirlink
        if(dir_link(ip, ".", ip->inum) < 0 ||
           dir_link(ip, "..", dp->inum) < 0)
            KERN_PANIC("create dots");
    }

    // FIXED: dir_link instead of dirlink
    if(dir_link(dp, name, ip->inum) < 0)
        KERN_PANIC("create: dir_link");

    inode_unlockput(dp);
    return ip;
}
```

```
void sys_open(tf_t *tf)
{
    char path[128];
    int fd, omode;
    struct file *f;
    struct inode *ip;
    unsigned int path_len;
    static int first = TRUE;
    if (first) {
        first = FALSE;
        log_init();
    }
    path_len = syscall_get_arg4(tf);
    path_len = path_len < 128? path_len: 127;
    pt_copyin(get_curid(), syscall_get_arg2(tf), path, path_len);

    path[path_len] = '\0';
    omode = syscall_get_arg3(tf);

    if (!path)
        KERN_PANIC("sys_open: no path");

    if(omode & O_CREATE){
        begin_trans();
        ip = create(path, T_FILE, 0, 0);
        commit_trans();
        if(ip == 0){
            syscall_set_retvall(tf, -1);
            syscall_set_errno(tf, E_CREATE);
            return;
        }
    } else {
        if((ip = namei(path)) == 0){
            syscall_set_retvall(tf, -1);
            syscall_set_errno(tf, E_NEXIST);
            return;
        }
        inode_lock(ip);
        if(ip->type == T_DIR && omode != O_RDONLY){
            inode_unlockput(ip);
            syscall_set_retvall(tf, -1);
            syscall_set_errno(tf, E_DISK_OP);
            return;
        }
    }

    if((f = file_alloc()) == 0 || (fd = fdalloc(f)) < 0){
        if(f)
            file_close(f);
        inode_unlockput(ip);
        syscall_set_retvall(tf, -1);
        syscall_set_errno(tf, E_DISK_OP);
        return;
    }
    inode_unlock(ip);

    f->type = FD_INODE;
    f->ip = ip;
    f->off = 0;
    f->readable = !(omode & O_WRONLY);
    f->writable = (omode & O_WRONLY) || (omode & O_RDWR);
    syscall_set_retvall(tf, fd);
    syscall_set_errno(tf, E_SUCC);
}
```

cat

mcertikos/kern/fs/sysfile.c

```
void sys_read(tf_t *tf)
{
    Implement with Codex
    //TODO
    int fd;
    unsigned int user_buf;
    unsigned int n;
    unsigned int pid = get_curid();
    struct file *fp;
    int size_read;
    spinlock_acquire(&buf_lock);
    fd = (int)syscall_get_arg2(tf);
    user_buf = syscall_get_arg3(tf);
    n = syscall_get_arg4(tf);
    if(fd < 0 || fd >= NOFILE || n < 0){
        KERN_INFO("fd illegal\n");
        syscall_set_errno(tf, E_BADF); // BAD FILE DESCRIPTOR
        syscall_set_retval(tf, -1);
        spinlock_release(&buf_lock);
        return ;
    }
    if (user_buf < VM_USERLO || user_buf + n > VM_USERHI || n > sizeof(kern_buf)){
        KERN_INFO("user_buf illegal\n");
        KERN_INFO("user_buf = %u, n = %u, VM_USERLO = %u, VM_USERHI = %u\n", user_buf, n, VM_USERLO, VM_USERHI);
        syscall_set_errno(tf, E_INVALID_ADDR);
        syscall_set_retval(tf, -1);
        spinlock_release(&buf_lock);
        return ;
    }
    fp = tcb_get_openfiles(pid)[fd];
    if(fp == 0 || fp->type == FD_NONE){
        if (fd == 0) {
            unsigned int i;
            for (i = 0; i < n; i++) {
                char c;
                while ((c = cons_getc()) == 0) {
                    ;
                }
                kern_buf[i] = c;
            }
            pt_copyout(kern_buf, get_curid(), user_buf, n);
            syscall_set_retval(tf, n);
            syscall_set_errno(tf, E_SUCC);
            spinlock_release(&buf_lock);
            return;
        }
        syscall_set_retval(tf, -1);
        syscall_set_errno(tf, E_BADF);
        spinlock_release(&buf_lock);
        return ;
    }
    size_read = file_read(fp, kern_buf, n);
    if(size_read > 0)
        || pt_copyout(kern_buf, get_curid(), user_buf, size_read);
    syscall_set_retval(tf, size_read);
    syscall_set_errno(tf, E_SUCC);
    spinlock_release(&buf_lock);
    return ;
}
```

echo

mcertikos/kern/fs/sysfile.c

```
void sys_write(tf_t *tf)
{
    Implement with Codex
    //TODO
    int fd;
    unsigned int user_buf;
    unsigned n;
    unsigned int pid = get_curid();
    struct file * fp;
    int size_write;
    spinlock_acquire(&buf_lock);
    fd = (int)syscall_get_arg2(tf);
    user_buf = syscall_get_arg3(tf);
    n = syscall_get_arg4(tf);
    if(fd < 0 || fd >= NOFILE || n < 0){
        syscall_set_errno(tf, E_BADF); // BAD FILE DESCRIPTOR
        syscall_set_retval(tf, -1);
        spinlock_release(&buf_lock);
        return ;
    }
    if (user_buf < VM_USERLO || user_buf + n > VM_USERHI || n > sizeof(kern_buf)){
        syscall_set_errno(tf, E_INVALID_ADDR);
        syscall_set_retval(tf, -1);
        spinlock_release(&buf_lock);
        return ;
    }
    fp = tcb_get_openfiles(pid)[fd];
    if(fp == 0){
        if (fd == 1) {
            unsigned int i;
            pt_copyin(pid, user_buf, kern_buf, n);
            for (i = 0; i < n; i++) {
                cons_putc(kern_buf[i]);
            }
            syscall_set_retval(tf, n);
            syscall_set_errno(tf, E_SUCC);
            spinlock_release(&buf_lock);
            return;
        }
        syscall_set_retval(tf, -1);
        syscall_set_errno(tf, E_BADF);
        spinlock_release(&buf_lock);
        return ;
    }
    pt_copyin(pid, user_buf, kern_buf, n);
    size_write = file_write(fp, kern_buf, n);
    syscall_set_retval(tf, size_write);
    syscall_set_errno(tf, E_SUCC);
    spinlock_release(&buf_lock);
    return ;
}
```

cp

user/shell/shell.c

```
int shell_cp(int argc, char** argv)
{
    char * src_path, * dest_path;
    if(argc < 3){
        printf("cp: too few arguments.\n");
        return 0;
    }
    else if(argc > 4){
        printf("cp: too many arguments.\n");
        return 0;
    }
    if(argc == 3){
        // ordinary cp
        src_path = argv[1];
        dest_path = argv[2];
        _shell_cp(dest_path, src_path, 0);
        return 0;
    }
    else{// recursive
        if(strcmp(argv[1], "-r")){
            printf("cp: invalid option. try '-r' ?\n");
            return 0;
        }
        // recursive cp
        src_path = argv[2];
        dest_path = argv[3];
        _shell_cp(dest_path, src_path, 1);
        return 0;
    }
}
```

mv

user/shell/shell.c

```
int shell_mv(int argc, char** argv)
{
    if(argc != 3){
        printf("mv: argument invalid.\n");
        return 0;
    }
    char* src = argv[1];
    char* dest = argv[2];
    if(!is_file_exist(src)){
        printf("mv: source file %s does not exist.\n", src);
        return 0;
    }
    if (is_dir(src)) {
        if (is_file_exist(dest) && !is_dir(dest)) {
            printf("mv: cannot move a dir to a file\n");
            return 0;
        }
        _shell_cp(dest, src, 1);
        _shell_rm(src, 1);
    }else {
        _shell_cp(dest, src, 1);
        _shell_rm(src, 0);
    }
    return 0;
}
```

rm

user/shell/shell.c

```
int shell_rm(int argc, char** argv)
{
    int isRecursive;
    int pathIdx;
    if (argc == 1) {
        printf("Too few arguments.\n");
        return 0;
    }

    if(!strcmp(argv[1], "-r")){
        isRecursive = 1;
        pathIdx = 2;
    }else{
        isRecursive = 0;
        pathIdx = 1;
    }
    if(pathIdx >= argc){
        printf("rm: no path argument.\n");
        return 0;
    }
    for (; pathIdx < argc; pathIdx++) {
        char * path = argv[pathIdx];
        if(!is_file_exist(path)){
            printf("rm: can not remove %s: is not a file or directory.\n", path);
            continue;
        }
        _shell_rm(path, isRecursive);
    }
    return 0;
}
```



THANK YOU